

ImagiGen : Converting Text and Images into a Visual Storytelling Experience

Dakshata Panchal¹, Shruti Humbar², Daniel George³

¹ St. Francis Institute of Technology, Mumbai, India

² St. Francis Institute of Technology, Mumbai, India

³ St. Francis Institute of Technology, Mumbai, India

Abstract—This research delves into the pursuit of generating realistic images from textual descriptions, a compelling yet challenging task within current AI systems. Although existing technologies fail to achieve this goal, recent advances in recurrent neural networks have demonstrated proficiency in learning discriminative text features. Additionally, DC-GAN has shown promise in generating highly realistic images across specific categories such as faces, album covers, and interiors. In this study, we introduce a novel deep architecture and GAN formulation aimed at bridging these text and image-modeling advances. Here approach is centered on translating textual concepts into vivid visual representations, effectively converting characters into pixelated images. Through rigorous experimentation, we showcase the ability of our model to produce credible images of flowers from intricate textual descriptions. This work represents a significant step toward achieving the synthesis of detailed images solely from text, offering insights into the convergence of text and image modelling within the realm of artificial intelligence.

Index Terms—Deep Convolutional Generative Adversarial Networks, ImagiGen, Text-to-Image Generation, Generative Adversarial Networks, Image Synthesis

102-flower dataset, each accompanied by descriptive captions, was curated. Subsequently, various preprocessing tasks were conducted, including image resizing and conversion of captions into numerical embeddings using word embeddings. The model comprises a generator responsible for producing images based on random seed vectors and caption embeddings, along side a discriminator tasked with distinguishing between authentic and fabricated images. Throughout the training process, optimization techniques such as the Adam optimizer were employed to refine the performance of both the generator and discriminator. Through this approach, the generator's ability to deceive the discriminator by generating images that closely resemble real ones was demonstrated, while the discriminator effectively discerns between authentic and generated images. Ultimately, the system transforms textual descriptions into images illustrating the evolution of flowers based on the provided input."

I. INTRODUCTION

"Recent advancements in AI, particularly in natural language processing and computer vision, have propelled text-to-image creation forward. This field seeks to seamlessly translate verbal descriptions into lifelike visuals, solely based on textual inputs. The capacity to transform textual semantics into visually appealing visuals has enormous potential in a variety of disciplines, including design, content development, and augmented reality applications. The automatic production of realistic images from written descriptions has a wide range of possible applications, including image editing, video games, and accessibility. This research has a goal of text-to-image creation is to produce realistic visuals that match the provided text in terms of semantic consistency. The requirement for many high-quality text-image pairs is one of the main obstacles in training text-to-image generating algorithms. Thus, in order to address this problem, GAN networks will be used, as stated by, which is exactly what we will do in our work. In Imagigen, a comprehensive process will be developed for generating flower images from textual descriptions. Initially, a Oxford

II. RELATED WORK

mage synthesis from text descriptions is a challenging yet developing area of computer vision research. To handle this challenge, several strategies have been presented, each with its distinct contributions and approaches. [1] proposed the Semantic-Spatial Aware GAN framework, which focuses on matching produced images with text descriptions at both the holistic and fine-grained levels, solving regional consistency issues. Similarly, [2] presented the Dynamic Memory Generative Adversarial Network (DM-GAN) to improve ambiguous initial image contents, focusing on the dynamic selection of critical text information for accurate image production. In their FuseDream pipeline, [3] coupled CLIP representations with off-the-shelf GANs in an original way, emphasizing optimization in the GAN space using creative ways to create varied, high-quality images. [4] addressed the aspect information difficulty by developing the Dynamic Aspect-aware GAN (DAE-GAN), which incorporates complete text representations at several granularities and refines images at the aspect and local levels. Moreover, [5] introduced the Cycle-consistent Inverse GAN

(CI-GAN), which unifies text-guided image creation and manipulation tasks through cycle-consistency training and latent space semantics exploration.[6] suggested a context-aware technique that separates foreground and background using a conditional VAE-GAN structure, hence improving text-image alignment. [7] used semantic disentanglement and distribution normalization modules to improve image creation from text and introduced distribution regularization in GANs. [8] concentrated on criminal face creation, using GANs to turn textual descriptions of facial characteristics into realistic human features. [9] introduced DF-GAN, which addresses entanglement difficulties in generator designs while also increasing semantic consistency and deepening text-image fusion for improved image synthesis. [10] presented CogView, a text-to-image synthesis system that combines a 4-billion-parameter Transformer with a VQ-VAE tokenizer, displaying adaptability across several downstream applications and delivering state-of-the-art performance in text-to-image synthesis.

III. BACKGROUND

The GAN system consists of two different networks: the generator and the discriminator. The generator creates synthetic samples by extracting random noise from a latent space, while the discriminator acts as a binary classifier and distinguishes between real and fake samples. When presented with a fake sample produced by the generator, the discriminator attempts to correctly identify it as fake, while the generator strives to produce samples that fool the discriminator into thinking they are real. This contradictory interaction forms the core of the GAN system.

The optimization process involves a min-max objective function, where the generator aims to minimize the objective function while the discriminator tries to maximize it. Specifically, the discriminator tries to drive the probability of classifying the generated samples ($D(G(z))$) towards 0 and thus maximize $(1-D(G(z)))$, while the generator tries to drive the probability of $D(G(z))$ towards 1 and thus minimize $(1-D(G(z)))$.

This min-max formulation of the objective function achieves a global optimum when the data distribution matches the model distribution. This means that the model has effectively learned the underlying data distribution in the training data set.

IV. METHOD

GAN models are very good at generating random images, but it is extremely difficult to control GAN to generate the images we are interested in. ImagiGen's methodology revolves around the strategic use of the DCGAN architecture, which explicitly integrates convolutional layers and convolutional transpose layers in the discriminator and generator networks, respectively. This methodology leverages the remarkable advances in the field of deep learning architectures, particularly in the area of generative adversarial networks ,

to facilitate the synthesis of visually compelling images from textual descriptions.

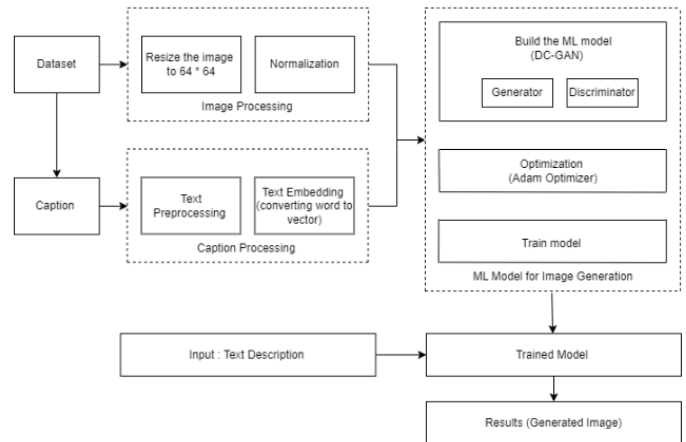


Fig. 1. Architecture of System

A. Dataset

The first step in ImagiGen involves preparing and preprocessing the dataset. We use the 102-flowers dataset [2], a widely used benchmark dataset that contains images of flowers with the corresponding text labels. This dataset serves as the basis for training our text-to-image synthesis model. The flower dataset contains 102 categories of flower images. Each category consists of 40-258 images. The total number of flower image-description pairs used in this project is over 7000. Each image is annotated with captions.

B. Image Processing

To enable efficient model training and ensure uniformity across the dataset, we resize all images to a standardized resolution. By resizing the images to a uniform dimension, typically 128*128 pixels, we reduce computational complexity and promote uniform feature extraction in the subsequent stages of the synthesis process. Before feeding the images into the DC-GAN model, we normalize the pixel values to a standardized range, usually [0, 1]. Normalization ensures that the input data corresponds to a common scale, which enables more stable and efficient model training.

C. Caption Processing

In the ImagiGen framework, text preprocessing is crucial to ensure the quality and relevance of textual descriptions guiding image synthesis. Word embedding, particularly utilizing the GloVe 6B 300D model, plays a pivotal role in bridging the semantic gap between textual descriptions and image representations. By encoding words into dense vector representations, GloVe captures semantic relationships and contextual information.

The GloVe model operates as an unsupervised learning

algorithm, leveraging global word co-occurrence statistics to learn vector representations for words. Unlike traditional methods, GloVe optimizes a global objective function that captures both local and global word-word relationships. These pre-trained GloVe embeddings enrich the semantic representation of textual descriptions in our ImagiGen framework, facilitating a more coherent image synthesis process.

D. DCGAN Network

A DCGAN is a direct extension of the GAN algorithm, with the exception that it explicitly uses convolutional layers and convolution-transposed layers in the discriminator and generator, respectively. The generator consists of convolutional transpose layers, batch norm layers and ReLU activations. The input is a latent vector z drawn from a standard normal distribution and the output is a $3 \times 64 \times 64$ RGB image. The strided conv transpose layers allow the latent vector to be converted into a volume that has the same shape as an image. The discriminator consists of stepped convolutional layers, batch norm layers and LeakyReLU activations. The input is a $3 \times 64 \times 64$ input image and the output is a scalar probability that the input comes from the real data distribution.

- Generator:

The generator, G , maps the latent space vector (z) to the data space. It uses a series of stepped two-dimensional convolutional transpose layers, each paired with a 2D batch norm layer and a ReLU activation. The output of the generator is passed through a tanh function to return it to the input data range $[-1,1]$. Batch normalization layers are used after the conv transpose layers to support the flow of gradients during training. An image of the generator from the DCGAN paper is shown below.

- 1) Project and Reshape: This block probably takes a random noise vector (seed vector) as input and reshapes it into a specific format suitable for the first convolutional layer. The text "100" next to the input could indicate the dimension (size) of the noise vector.
- 2) Conv Layers (CONV 1, CONV 2, CONV 3, CONV 4): These are the core layers of the generator. They are convolutional layers that gradually increase the spatial resolution and channel depth of the feature maps. Each convolutional layer applies filters to the output of the previous layer, extracting higher-level features and building the image step-by-step. The numbers indicated next to "CONV" (like "1024, 512, 256, 64") probably represent the number of filters used in each convolutional layer. More filters allow the network to capture a greater variety of features.
- 3) Batch Norm: These layers are inserted after some convolutional layers (e.g., after CONV 2 and CONV 3). Batch normalization helps to improve the training speed and stability of the network by normalizing the activations of the previous layer.

- 4) ReLU: This activation function (displayed after each "CONV" and "Batch Norm") introduces a non-linearity into the network. It allows the network to learn complex relationships between the input noise vector and the generated image.
- 5) Output ($G(z)$): This is the final output of the generator, which is a 3-channel RGB image with the same resolution (probably 64×64 based on the diagram) as the training images.

- Discriminator:

The discriminator- D , is a binary classification network that takes an image as input and outputs a scalar probability that the input image is real. D processes the input image through a series of Conv2d, BatchNorm2d, and LeakyReLU layers and outputs the final probability through a sigmoid activation function.

Components:

- 1) Input image: The discriminator takes either a real image from the training set or an image generated by the generator as input.
- 2) Conv Layers: The image is passed through a series of convolutional layers that extract features relevant for distinguishing real from fake images.
- 3) Leaky ReLU: After each convolution, a Leaky ReLU activation function introduces nonlinearity into the network. It enables the network to learn complex relationships between the input noise vector and the generated image.
- 4) Flatten: The final feature maps from the convolutional layers are flattened into a 1D vector.
- 5) Dense layer: The flattened vector is fed into a fully linked layer that integrates the extracted features.
- 6) Sigmoid activation: The last layer uses a sigmoid activation function to output a probability value between 0 and 1. A value closer to 1 means that the discriminator believes that the image is genuine, while a value closer to 0 means that it is fake (generated).

- Optimization:

Adam optimizes the learning process by adjusting the model parameters based on the gradients of the loss function. It adapts the learning rates for each parameter individually, enabling efficient convergence during training. By utilizing Adam optimization, ImagiGen improves the stability and effectiveness of the training process, leading to the generation of high-quality visual narratives.

- Training:

- 1) Train the Generator
 - Upsampling (Upsampling_{1,2,3,4} layers): Upsampling layers are integral components in convolutional neural networks (CNNs), especially in tasks such as image generation and semantic segmentation. They are employed to augment the spatial dimensions of an input tensor, effectively enlarging it to a higher resolution. Various types of upsampling

layers exist, including nearest neighbor upsampling, bilinear upsampling, and transposed convolution (deconvolution or fractionally strided convolution).

- Batch normalization (BatchNorm1,2,3,4) is a pivotal technique in training deep neural networks, particularly in the context of Generative Adversarial Networks (GANs). Its primary purpose is to normalize the input of each layer to have a mean of zero and a standard deviation of one. This normalization is applied to mini-batches of data and serves several crucial roles in network training.

- LeakyReLU : Leaky ReLU addresses a limitation of the popular ReLU activation function by introducing a slight modification for negative inputs. While ReLU excels in computational efficiency, it suffers from the "dying ReLU" problem, where neurons cease to learn due to constant negative inputs. Leaky ReLU tackles this by allowing a small, positive gradient to flow for negative values through a user-defined coefficient. This prevents neurons from becoming inactive and improves the network's ability to handle noisy data, potentially leading to better overall performance. Its value ranges from -1,1.

- Tanh: Tanh is utilized as the final activation function in the generator model for its suitability in producing images with realistic pixel values. With an output range spanning from -1 to 1, Tanh aligns well with the normalized input data, ensuring that the generated images maintain appropriate pixel intensity levels. Its smooth and continuous nature enables seamless transitions between pixel values, contributing to visually appealing images. Moreover, Tanh's zero-centered output aids in stabilizing training by mitigating issues like vanishing gradients. By effectively squashing input values into a bounded range, Tanh helps capture intricate patterns and details in the generated images, facilitating the creation of high-quality outputs.

Generator Processing Steps:

- a) Input Preparation and Integration: The generator function begins by accepting two inputs, a noise vector and an embedding vector of 300 dimension representing textual information. These inputs are crucial as they provide the necessary information for generating realistic images conditioned on specific textual descriptions.
- b) Convolutional layer 1: In the first layer, a dense neural network layer is employed to process the embedding vector. This layer serves to transform the textual embedding into a dense representation, facilitating its integration with the random noise vector. The merging layer concatenates the processed embedding with the noise vector, enabling the model to incorporate both sources

of information effectively.

- c) Convolutional layer 2: Following the merging layer, a dense layer coupled with a reshape operation prepares the data for subsequent convolutional operations. This dense layer further refines the merged information, while the reshape operation organizes the data into a suitable format for convolutional processing. Subsequently, a series of upsampling and convolutional layers are applied to gradually increase the spatial resolution of the feature map. The upsampling layers expand the feature map dimensions, while the convolutional layers learn to fill in missing details and capture intricate patterns.
 - d) Convolutional layer 3: As the generator progresses through these layers, batch normalization is employed to stabilize and accelerate the training process by normalizing the activations of the preceding layers. This normalization helps mitigate issues such as vanishing gradients and enables smoother convergence during training. Additionally, the LeakyReLU activation function is applied after each convolutional layer to introduce non-linearity into the network. The LeakyReLU function ensures that the model can learn complex relationships between the input noise vector, textual embedding, and the generated image.
 - e) Convolutional layer 4: Towards the end of the generator architecture, the final upsampling and convolutional layers prepare the feature map for image generation. These layers upscale the feature map to match the desired output size and convolve it to produce the final image. Finally, the output convolutional layer generates the RGB image by applying the appropriate number of filters. The activation function, Tanh, is utilized to scale the output image to the range [-1, 1], ensuring compatibility with image data.
- a) Train the Discriminator
 - Dropout: To prevent overfitting during training, the discriminator model incorporates dropout layers after several convolutional layers. Dropout randomly deactivates a set proportion of neurons within a layer for each training iteration. These inactive neurons don't contribute to the forward pass calculations and are excluded from back-propagation updates. However, their weights are still adjusted using a scaling factor to account for the reduced number of active neurons. This technique encourages the network to develop robust features that are not dependent on specific neuron activations, ultimately improving the model's ability to generalize to unseen data.
 - Sigmoid: The sigmoid activation function is

utilized in the final layer of the discriminator network. Its primary role is to produce an output value within the range $[0, 1]$, by squashing the output of the last dense layer into the range $[0, 1]$, it effectively interpreting it as a probability score. A value close to 1 indicates high confidence that the input image is real, while a value close to 0 suggests high confidence that the input image is generated. This functionality aligns with the discriminator's objective, which is to classify input images as real or generated. By applying the sigmoid activation function, the discriminator can provide a probabilistic assessment of the authenticity of the input image, facilitating adversarial training with the generator network.

Discriminator Processing Steps:

- i) Convolutional layer 1: The first layer of the discriminator network initiates the feature extraction process by convolving learnable filters with the input image. With 32 filters of size 4×4 , this layer analyzes local patterns within the image. Employing a stride of 2, the convolution operation progresses by two pixels in both horizontal and vertical directions, effectively downsampling the feature maps. The LeakyReLU activation function adds non-linearity to the output of the convolution, enabling the model to capture complex patterns and structures in the input image.
- ii) Convolutional layer 2: Following the initial feature extraction, dropout regularization is applied to mitigate overfitting, randomly deactivating 25
- iii) Convolutional layer 3: To extract higher-level features, dropout regularization is again employed for robustness against overfitting. The third Conv2D layer utilizes 128 filters of size 4×4 to analyze the feature maps generated by the preceding layers. With a stride of 2, this layer continues the downsampling process, capturing more abstract representations of the input image. Batch normalization normalizes the activations, promoting stable optimization. LeakyReLU activation introduces non-linearity, allowing the model to discern complex patterns.
- iv) Convolutional layer 4: Further enhancing feature extraction, dropout regularization is once again applied. The fourth Conv2D layer employs 256 filters of size 4×4 to delve deeper into the features learned by the previous layers. With a stride of 2, this layer further reduces the spatial dimensions of the feature maps, extracting more abstract representations. Batch normalization standardizes the activations, ensuring consistent training dynamics. LeakyReLU activation fosters non-linear transformations, aiding the model in capturing intricate patterns.
- v) Embedding Concatenation: Transitioning to incorporating additional information, the input embedding undergoes processing to match the dimensions of the convolutional feature maps. LeakyReLU activation is applied to introduce non-linearity into the embedding. Reshaping the embedding aligns its spatial dimensions with those of the feature maps, facilitating concatenation. The concatenated feature maps and embeddings enrich the model's representation with additional contextual information, enhancing its discriminative capabilities.
- vi) Convolutional layer 5: To further refine feature representation, dropout regularization is applied for regularization. The fifth Conv2D layer applies 512 filters of size 4×4 to the concatenated feature maps, extracting discriminative features. This layer continues to analyze the combined embeddings and feature maps, capturing nuanced patterns. Batch normalization maintains stable activations, facilitating efficient training. LeakyReLU activation enables non-linear transformations, enabling the model to discern intricate patterns.
- vii) Final Layers (Flatten and Dense): Concluding the discriminator architecture, dropout regularization is employed for robustness. The feature maps are flattened into a 1-dimensional vector to prepare for the final classification. A dense layer produces a single output unit with sigmoid activation, representing the discriminator's prediction of the input image's authenticity.

E. Input

User Interacts with the system to provide input text descriptions. The processed text data is used to generate images using the pre-trained DC-GAN model.

F. Result

This marks the point of end-user interaction, where users engage with the images reflecting the user's narrative vision.

V. EXPERIMENT

A. Quantitative Results

After an extensive 24-hour training period spanning approximately 450 epochs, the Image Generator Model

achieved significant progress at a resolution of 64*64 pixels. The model, incorporating both Discriminator and Generator networks crucial for GAN-based text-to-image generation, exhibited visible advancement by the 438th epoch. At this point, the Generator loss stood at 1.3284586668014526, and the Discriminator loss at 1.2313429117202759, indicating nearing equilibrium between the networks. The diminishing generator loss signifies enhanced capability in generating realistic images from textual prompts. In contrast, the higher loss of the discriminator, following the adversarial training principle, underscores the efficacy of the network in distinguishing genuine images from synthesized ones:

Table1. Experimental analysis on 102-flowers dataset with 64*64 resolution:

Epochs	Generator Loss	Discriminator Loss
100	0.9697360992431641	1.3281246423721313
250	1.0975538492202759	1.299974799156189
438	1.3284586668014526	1.2313429117202759

For the 128*128 resolution, training lasted 48 hours across 1000 epochs, yielding a generator loss of 0.6933209896087646 and a discriminator loss of 1.3896995782852173. Analyzing these outcomes, it's evident that the model trained on 64*64 resolution images exhibited faster convergence and lower loss values compared to the model trained on 128*128 resolution images. This implies that the 64*64 resolution model reached a more stable state in terms of image generation within a shorter training period. Table2. Experimental analysis on 102-flowers dataset with 128*128 resolution:

Epochs	Generator Loss	Discriminator Loss
100	0.6947812438011169	1.3897820711135864
500	0.6933603882789612	1.3906561136245728
1000	0.6933209896087646	1.3896995782852173

B. Qualitative Results

The textual description accompanying the flower image emphasizes the distinct visual attributes. Subsequently, upon examining the test output result, it reveals a multitude of smaller images. These images are the generated outputs produced by the model in response to the provided textual description. The presence of numerous smaller images suggests the model's attempts to interpret and generate various visual representations corresponding to the textual description of a purple-coloured flower with oval-shaped petals. This observation underscores the model's efforts in generating diverse outputs to encapsulate the potential variations and nuances inherent in the given textual input, aiming to produce a range of plausible floral images that align with the described

characteristics. Further evaluation and analysis of these outputs would provide deeper insights into the model's ability to interpret textual descriptions.

VI. CONCLUSION

This research leverages recent advancements in deep convolutional generative adversarial networks (DC-GANs) to generate highly realistic images within specific categories. By integrating these innovations, the study bridges the gap between text and image modeling, leading to the development of a robust deep architecture and GAN framework. At 64x64 resolution, the model's accuracy at epoch 438 shows the generator and discriminator losses stabilizing at approximately 1.32 and 1.231, respectively, indicating convergence. However, further training or adjustments may be required to enhance the image quality.

REFERENCES

- [1] Liao, W., Hu, K., Yang, M. Y., and Rosenhahn, B., "Text to image generation with semantic-spatial aware gan," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 18187-18196, 2022.
- [2] Zhu, M., Pan, P., Chen, W., and Yang, Y., "Dm-gan: Dynamic memory generative adversarial networks for text-to-image synthesis," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 5802-5810, 2019.
- [3] Liu, X., Gong, C., Wu, L., Zhang, S., Su, H., and Liu, Q., "sedream: Training-free text-to-image generation with improved clip + gan space optimization," in *arXiv preprint arXiv:2112.01573*, 2021.
- [4] Ruan, S., Zhang, Y., Zhang, K., Fan, Y., Tang, F., Liu, Q., and Chen E., "Dae-gan: Dynamic aspect-aware gan for text-to-image synthesis," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 13960-13969, 2021.
- [5] Wang, H., Lin, G., Hoi, S. C., and Miao, C., "Cycle-consistent inverse GAN for text-to-image synthesis," in *Proceedings of the 29th ACM International Conference on Multimedia*, pp. 630-638, 2021.
- [6] Hang, C., and Peng, Y., "Stacking VAE and GAN for context-aware text-to-image generation," in *018 IEEE Fourth international conference on multimedia big data (BigMM)*(pp. 1-5), 2018.
- [7] Tan H., Liu X., Yin B., and Li X., "DC-GAN: Distribution regularization for text-to-image generation", in *Transactions on Neural Networks and Learning Systems*, Vol. 8, pp. 153113-153122, 2022.
- [8] Mahajan, S., Nighrunkar, M., Kulkarni, A., Joshi, A., and Sawant, S., "Theft detection system using cGAN approach. In AIP Conference Proceedings", in *AIP Publishing*, Vol. 2745, No. 1, 2023.
- [9] Tao M., Tang H., Wu F., Jing X. Y., Bao B. K., and Xu, C., "Df-gan: A simple and effective baseline for text-to-image synthesis", in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (pp. 16515-16525), 2022.
- [10] Ding, M., Zheng, W., Hong, W., and Tang, J., "Cogview2: Faster and better text-to-image generation via hierarchical transformers", in *Neural Information Processing Systems*, 35, 16890-16902, 2022.